

[< Go to the original](#)

The Hidden Costs of LangChain, CrewAI, PydanticAI and Others: Why Popular AI Frameworks Are Failing...

After 15 years in software development and countless hours wrestling with AI frameworks, I had reached a breaking point. The promise of...



Follow



AI Advances a11y-light ~9 min read · July 13, 2025 (Updated: July 13, 2025) · Free: No

After 15 years in software development and countless hours wrestling with AI frameworks, I had reached a breaking point. The promise of "production-ready" AI development tools has become a minefield of abstractions, vendor lock-ins, and overhyped capabilities. Let me share why popular frameworks like LangChain, CrewAI, and even the newer PydanticAI fall short, and why I created Atomic Agents as the solution we actually need.

The LangChain Labyrinth: When Abstractions Attack

LangChain quickly became a go-to, and its modularity is often touted as a strength. However, this modularity can quickly devolve into a complex setup and configuration nightmare. One of the biggest pain points is the lack of control over autonomous agents. The framework often makes hidden calls to LLMs, chaining requests in ways that aren't transparent. The result? Unpredictable costs, inefficient execution, and workflows that can become unnecessarily long and difficult to debug.

The Hidden Cost Problem

Here's a real scenario from my own experience: I once had a simple document analysis task balloon into hundreds of dollars in API costs because LangChain was making multiple hidden LLM calls I couldn't see or control. The framework's layers of abstraction made it nearly impossible to understand what was happening under the hood.

Documentation: A Maze Without a Map

LangChain's documentation has been a point of contention for many users. Compared to some competing frameworks, the quality of LangChain's documentation has been criticized as subpar, with reports of: Incomplete guidance and unclear explanations, Outdated information and broken links, Misleading code examples

The documentation reads like it was written by people who've never actually built a production system. You'll find yourself diving through GitHub issues and Discord channels just to understand basic functionality.

Real Companies Are Moving Away

found that LangChain's abstractions were too inflexible for more complex agent architectures (like agents spawning sub-agents or specialized agents coordinating). After growing frustrations, Octomind's team decided to remove LangChain entirely in 2024. The result? "Once we removed it... we could just code," they report, noting that no longer being constrained by LangChain made their team far more productive.

CrewAI: The Overpromise Express

CrewAI offers a more structured approach to multi-agent systems, focusing on role-playing agents that collaborate on tasks. This is great for certain use cases, like research teams or project management. However, this structured approach can also be its Achilles' heel.

Rigidity Disguised as Structure

The framework can be rigid, making it difficult to dynamically adjust roles or delegate tasks mid-workflow. While it aims for simplicity, some users report inconsistent results for specific use cases and a need for significant performance tuning to achieve stability in complex tasks.

communicate? Good luck navigating their rigid crew-based programming model.

AutoGen: Microsoft's Half-Baked Solution

Microsoft's AutoGen empowers developers with multi-agent conversations and offers flexibility with human-in-the-loop problem-solving. It's powerful, especially for complex problem-solving and research tasks. However, it's not without its frustrations. A common complaint is that its documentation can be hard to navigate, with insufficient examples. Some developers report that certain features, like structured outputs, "flat out don't work" as expected.

Think about that for a moment: a Microsoft-backed project shipping with features that simply don't work as advertised. This should tell you everything about the current state of AI frameworks.

PydanticAI: New Player, Same Problems

When PydanticAI was announced, I had high hopes. Built by the team behind Pydantic (which powers much of the Python ecosystem), it promised to bring

Still in Beta, Still Breaking

PydanticAI is in beta. This article is based on version 0.0.13. Code examples may not work with future versions. Limitations that are mentioned may be lifted in future versions. This isn't just a version number — it's a warning. The framework is actively breaking APIs and making major changes. As they state in their roadmap, break APIs ASAP so we can get to a stable state ASAP is literally part of their philosophy.

Building production systems on a framework that openly admits to breaking changes? That's a recipe for maintenance nightmares I've seen too many times before.

The Logfire Lock-in

Here's where things get concerning. Pydantic Services, the company behind Pydantic, has raised a \$12.5m Series A in October 2024. This is great news for the project: funding pays for full time developers. It also raises the question of how Pydantic will make money, and the answer to that is Logfire subscriptions.

This creates an uncomfortable dynamic where the "best" experience requires their paid product.

Limited Production Features

Despite marketing itself as "production-grade," PydanticAI is missing crucial features:

- No built-in retry mechanisms (unlike Instructor, which it claims to improve upon)
- its validation and streaming modes have some rough edges
- The roadmap shows they're still figuring out fundamental architecture

The Real Problem: Developers Building for VCs, Not Users

The common thread across all these frameworks? They're built to impress investors, not solve real problems. A lot of money was put into these types of companies by VC and investors. A bubble of ignorance that now seems to have at least slightly burst.

buzzwords and demos rather than the boring but essential work of building reliable, maintainable systems.

Enter Atomic Agents: Built by Developers, for Developers

After wrestling with these frustrations, I created Atomic Agents with a simple philosophy: no magic, just solid engineering.

The Power of True Atomicity

By focusing on single-purpose building blocks ("atoms"), Atomic Agents allows you to fine-tune every part of your system. Swap out a web-scraper for a database query, or a local LLM for an OpenAI model — no extensive rewrites needed. Strict input/output schemas let you define precisely what you expect, ensuring consistent results instead of random structure issues or half-baked JSON.

Every component in Atomic Agents:

- Has a single, clear responsibility
- Can be tested in isolation

- Uses standard Python practices you already know

No Hidden Complexity

If you're a Python dev, it's just Python. You can step through with a debugger, log everything, or integrate these agents into standard web frameworks (Flask, FastAPI, Django, etc.) without friction.

When something goes wrong (and it will), you can set a breakpoint and see exactly what's happening. No diving through layers of abstraction or trying to decode what some "magic" orchestrator is doing.

Schema-First Development with Real Control

By leveraging Pydantic (yes, the same Pydantic that PydanticAI uses), we ensure type safety without the overhead. But unlike PydanticAI, we don't try to reinvent the wheel:

Copy

```
from atomic_agents.agents.base_agent import BaseIOSchema
from pydantic import Field
from typing import List
```

```

# Input schema for the QueryAgent.
instruction: str = Field(
    ...,
    description="A detailed instruction or request to generate search engine queries"
)
num_queries: int = Field(..., description="The number of search queries to generate")
# The output of one agent becomes the input of another
from web_search_agent.tools.searxng_search import SearxNGSearchTool
query_agent = BaseAgent(
    BaseAgentConfig(
        client=instructor.from_openai(openai.OpenAI()),
        model="gpt-4o-mini",
        system_prompt_generator=SystemPromptGenerator(
            background=[
                "You are an intelligent query generation expert."
            ],
            steps=[
                "Receive the instruction and the number of queries.",
                "Generate the queries in JSON format."
            ],
            output_instructions=[
                "Ensure each query is unique and relevant.",
                "Provide the queries in the expected schema."
            ],
        ),
        input_schema=QueryAgentInputSchema,
        output_schema=SearxNGSearchTool.input_schema # Direct chaining!
    )
)

```

Copy

```
# Run the query agent
query_result = query_agent.run(query_input)

# Apply your business logic
filtered_queries = []
for query in query_result.queries:
    # Check against your business rules
    if not contains_prohibited_terms(query):
        filtered_queries.append(query)

# Maybe log for compliance
audit_log.record_query(query, user_id, timestamp)

# Apply rate limiting
if len(filtered_queries) >= get_user_query_limit(user_id):
    break

# Only then pass to the search tool
search_input = SearxNGSearchTool.input_schema(queries=filtered_queries)
search_results = search_tool.run(search_input)
# More business logic after search
validated_results = validate_search_results(search_results)
enriched_results = add_metadata_from_database(validated_results)
# Finally pass to your QA agent
qa_input = QAAgentInputSchema(
    question=original_question,
```

```
qa_result = qa_agent.run(qa_input)
```

This is **real** control. You decide when to chain directly, when to insert business logic, when to branch conditionally. No framework magic dictating your flow.

Real Production Features

Unlike the frameworks above, Atomic Agents includes what you actually need:

1. **Context Providers:** Inject dynamic data into system prompts without hacks
2. **Memory Management:** Built-in conversation tracking that you control
3. **Tool Ownership:** Download tools into your codebase — you own them completely
4. **No Vendor Lock-in:** Use any LLM provider, any observability platform, any deployment method

The CLI That Actually Helps

The Atomic Assembler CLI provides complete control over your tools, allowing you to:

- Avoid dependency clutter: Install only the tools you need.
- Modify tools easily: Each tool is self-contained with its own tests.

Building Real Systems, Not Demos

Here's a complete example that shows the difference. While other frameworks would have you navigating through abstractions, with Atomic Agents it's just clear, testable code:

Copy

```
from atomic_agents.agents.base_agent import BaseAgent, BaseAgentConfig
from atomic_agents.lib.components.system_prompt_generator import SystemPromptGe
from atomic_agents.lib.components.agent_memory import AgentMemory
from atomic_agents.lib.base.base_io_schema import BaseIOSchema
from pydantic import Field
from typing import List
import instructor
import openai

# Step 1: Define what goes in and out (no magic!)
class ResearchInput(BaseIOSchema):
    """Input schema for research queries."""
    question: str = Field(..., description="Research question")
class ResearchOutput(BaseIOSchema):
    """Output schema for research results."""
    answer: str = Field(..., description="Researched answer")
    sources: List[str] = Field(..., description="Source URLs")
# Step 2: Configure your agent (you control everything)
agent = BaseAgent(
    config=BaseAgentConfig(
```

```
system_prompt_generator=SystemPromptGenerator(
    background=["You are a research assistant."],
    steps=["Search for information", "Synthesize findings"],
    output_instructions=["Provide sources for all claims"]
),
input_schema=ResearchInput,
output_schema=ResearchOutput,
memory=AgentMemory()
)
# Step 3: Use it (no hidden API calls or surprise costs)
result = agent.run(ResearchInput(question="What is quantum computing?"))
print(f"Answer: {result.answer}")
print(f"Sources: {result.sources}")
```

That's it. No magic. No hidden complexity. Just code you can understand, test, and maintain.

The Industry Is Waking Up

Posts and articles with titles like 'Why we no longer use LangChain' began appearing, reflecting real companies deciding to pull LangChain out of their stacks. [One analysis on Medium](#) noted that While some developers acknowledge

Developers are realizing that Developers use it to learn AI development, and for prototyping rather than for production due to bad code quality, and high component complexity.

The Choice Is Yours

You can continue wrestling with:

- LangChain's maze of abstractions and hidden costs
- CrewAI's rigid structures and broken promises
- AutoGen's non-functional features
- PydanticAI's beta instability and vendor lock-in

Or you can choose a framework that respects you as a developer and treats you as an adult.

If you're tired of wrestling with frameworks that fight you every step of the way, I urge you: Explore [Atomic Agents on GitHub](#).

The Future Is Atomic

The AI development landscape doesn't need more magic or AGI promises. It needs solid engineering principles applied to real problems. That's what Atomic Agents delivers.

Join us in building AI systems that are:

- **Predictable:** You know exactly what will happen
- **Maintainable:** Your future self will thank you
- **Testable:** Unit tests that actually work
- **Portable:** No vendor lock-in, ever
- **Flexible:** Chain directly or add business logic — your choice

Stop fighting your tools. Start building with Atomic Agents.

Ready to experience AI development without the frustration? Check out [Atomic Agents on GitHub](#) and join our [community on Reddit](#). Built by developers who've been burned by the alternatives — and learned from it.

Support the Author

If you found this article useful, please consider donating any appropriate amount to my PayPal.me tip jar!

Pay Kenny Vaneetvelde using PayPal.Me

Go to paypal.me/KennyVaneetvelde and type in the amount. Since it's PayPal, it's easy and secure. Don't have a PayPal...

paypal.com

Your support means the world and allows me to continue to spend time writing articles, making tutorials, ...

Thank you!

If you **loved** my content and want to **get in touch**, you can do so through **[LinkedIn](#)** or even feel free to reach out to me by email at **kenny@brainblendai.com**

Similarly, if you need an AI-driven project or prototype developed, please contact my agency: **[BrainBlend AI](#)** and we will make sure your project gets the quality

Freedium

ko-fi librepay patreon

You can also find me on [X/Twitter](#) or you can give me a [follow on GitHub](#) and check out and star any of my projects on there, such as [Atomic Agents](#)!



[#langchain](#) [#python](#) [#programming](#) [#technology](#) [#ai](#)